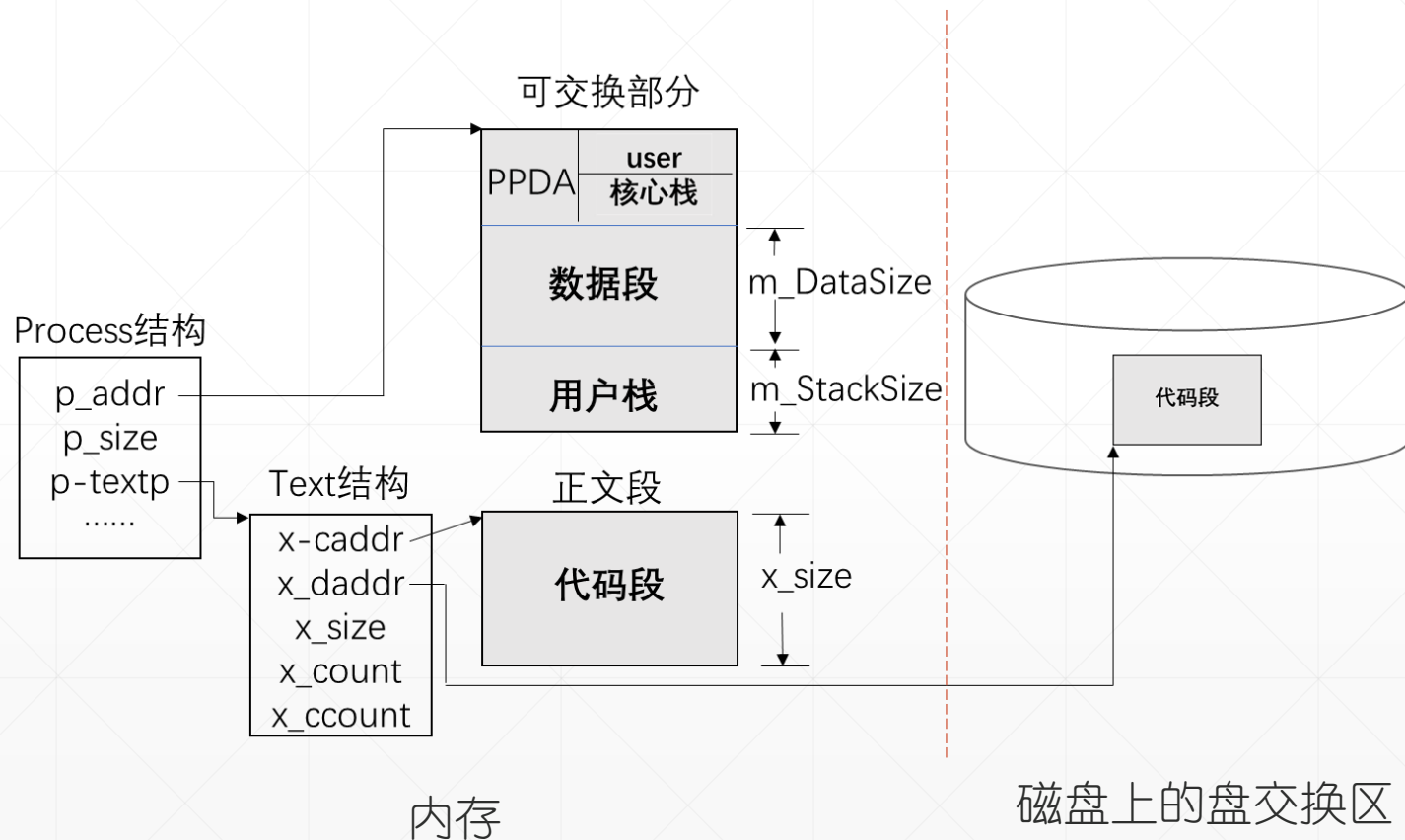


操作系统

第二章 进程管理

配图：Unix V6++进程图像，process结构和 user结构

Unix V6++ 进程图像



- **Process结构**存放在核心区中的Process数组。
- **共享正文段**，包括代码段和只读数据段 (Unix V6++ buggy, 把只读数据段放在了可交换区)，可与执行同一应用程序的其它进程共享。这是为了节省进程图像的内存消耗。
- **可交换部分**，包括PPDA区，数据段和用户栈。PPDA (Per Process Data Area) 区，4096字节，低地址端是user结构，高地址端是进程的核心栈。可交换部分禁止其它进程访问。



Class Process

{
public:

// 1、进程标识
short p_uid; // 用户标识
int p_pid; // 进程标识
int p_ppid; // 父进程标识

// 2、分配给正文段和可交换部分的物理内存

unsigned long p_addr; // 可交换部分起始地址
unsigned int p_size; // 可交换部分的长度 (以字节单位)
Text* p_textp; // 正文段的控制块

// 3、进程状态

ProcessState p_stat; // CPU调度状态
int p_flag; // 其它状态标识。SLOAD为1表示可交换部分和正文段在内存，
SSYS为1表示系统进程 (0#进程的SSYS标识为1)

// 4、进程优先数

int p_pri; // 进程优先数
int p_nice; // 用来计算进程优先数，这是基准量
int p_cpu; // cpu使用时长，用来计算进程优先数，实现时间片轮转调度

// 5、驻留时间，是一个计时器

int p_time; // 进程在盘交换区或内存的驻留时间

// 6、进程睡眠原因

unsigned long p_wchan; // 是一个内核变量的首地址

// 7、进程收到的信号

int p_sig;
unsigned long p_sigmap;

// 8、进程的终端

TTY* p_ttyp; // 键盘输入、屏幕输出

Class Text

{
public:

int x_daddr; // 共享正文段在盘交换区上的起始扇区号
unsigned long x_caddr; // 共享正文段的物理内存起始地址，单位字节
unsigned int x_size; // 共享正文段长度，单位字节
Inode* x_iptr; // 指向可执行文件的内存inode
unsigned short x_count; // 共享正文段的进程数
unsigned short x_ccount; // 共享该正文段且图像在内存的进程数

class User

{
public:

// 0、连接Process结构

Process* u_procp; // 虚地址

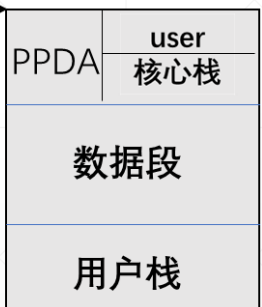
Process结构

p_addr
p_size
p_textp
.....

Text结构

x-caddr
x_daddr
x_size
x_count
x_ccount

可交换部分



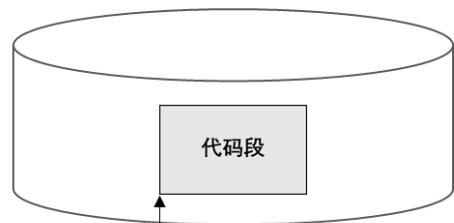
m_DataSize

m_StackSize

正文段

代码段

x_size





Class Process

{ public:

// 1、进程标识

short p_uid; // 用户标识
int p_pid; // 进程标识
int p_ppid; // 父进程标识

// 2、分配给正文段和可交换部分的物理内存

unsigned long p_addr; // 可交换部分起始地址
unsigned int p_size; // 可交换部分的长度 (以字节单位)
Text* p_textp; // 正文段的控制块

// 3、进程状态

ProcessState p_stat; // CPU调度状态
int p_flag; // 其它状态标识。SLOAD为1表示可交换部分和正文段在内存，SSYS为1表示系统进程 (0#进程的SSYS标识为1)

// 4、进程优先数

int p_pri; // 进程优先数
int p_nice; // 用来计算进程优先数，这是基准量
int p_cpu; // cpu使用时长，用来计算进程优先数，实现时间片轮转调度

// 5、驻留时间，是一个计时器

int p_time; // 进程在盘交换区或内存的驻留时间

// 6、进程睡眠原因

unsigned long p_wchan; // 是一个内核变量的首地址

// 7、进程收到的信号

int p_sig;
unsigned long p_sigmap;

// 8、进程的终端

TTY* p_ttyp; // 键盘输入、屏幕输出

Class Text

{ public:

int x_daddr; // 共享正文段在盘交换区上的起始扇区号
unsigned long x_caddr; // 共享正文段的物理内存起始地址，单位字节
unsigned int x_size; // 共享正文段长度，单位字节
Inode* x_iptr; // 指向可执行文件的内存inode
unsigned short x_count; // 共享正文段的进程数
unsigned short x_ccount; // 共享该正文段且图像在内存的进程数

class User

{ public:

// 0、连接Process结构

Process* u_procp; // 虚地址

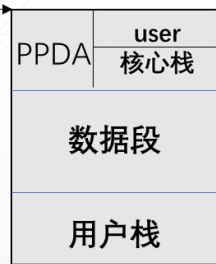
Process结构

p_addr
p_size
p-textp
.....

Text结构

x-caddr
x_daddr
x_size
x_count
x_ccount

可交换部分



m_DataSize

m_StackSize

正文段

代码段

x_size

代码段

tty输入缓存

tty输出缓存



```
class User
{
public:
```

```
// 0、连接Process结构
Process* u procp; // 虚地址
```

```
// 1、内存描述符
MemoryDescriptor u MemoryDescriptor;
```

```
// 2、进程切换时，用来保护 ESP、EBP
unsigned long u_rsav[2];
unsigned long u_ssav[2];
```

```
// 3、登记进程运行时长
int u_untime;    // 用户态运行时长
int u_stime      // 核心态运行时长
int u_cutime;    // 子进程用户态运行时长的总和
int u_cstime;    // 子进程核心态运行时长的总和
```

```
// 4、信号处理 与 被打断的系统调用
unsigned long u_signal[NSIG]; // 信号处理方式（函数）
unsigned long u_qsav[2];      // 信号接收进程应立即返回用户态执行信号处理
                                // 函数，这是被打断的系统调用的远跳转点
bool u_intflg; // 标识系统调用执行状态，1表示执行期间被信号打断，0表示未被打断
```

```
// 5、进程的用户标识
short u_uid;      // 有效用户ID
short u_gid;      // 有效组ID
short u_ruid;     // 真实用户ID
short u_rgid;     // 真实组ID
```

```
// 6、系统调用的入口参数 和 出错码
unsigned int*u_ar0;
int u_arg[5];
ErrorCode u_error; // 系统调用的出错码
```

```
class MemoryDescriptor
{
public:
```

```

PageTable* m_UserPageTableArray; // 进程的页表
// ----- 虚空间布局 -----
unsigned long m_TextStartAddress; // 代码段起始地址（虚地址）
unsigned long m_TextSize; // 代码段长度
unsigned long m_DataStartAddress; // 数据段起始地址（虚地址）
unsigned long m_DataSize; // 数据段长度
unsigned long m_StackSize; // 栈段长度（虚地址）

```

页表，地址映射关系

虚空间布局

